

Voxelizacion Paralela de Mallas Triangulares

Jose Francisco Paca Sotero
Integrantes del grupo

Computacion Paralela y Distribuida – 2026-I

Resumen

Este informe presenta una implementacion paralela de voxelizacion conservadora de mallas triangulares. La version principal usa MPI con particionamiento por triangulos, grillas locales por proceso y reduccion OR final. Para justificar la eleccion del paradigma, tambien se implementaron dos variantes OpenMP de memoria compartida: una con grilla privada por thread y reduccion OR, y otra con grilla compartida usando atomic OR. El analisis compara tiempo, speedup, eficiencia y overhead experimental para resoluciones moderadas.

1. Introduccion

La voxelizacion convierte una malla triangular en una grilla tridimensional binaria. En este proyecto, un voxel queda ocupado si pertenece a la caja envolvente discreta de al menos un triangulo. Esta aproximacion es conservadora y simple: puede marcar falsos positivos, pero mantiene una regla clara y paralelizable para construir una representacion volumetrica.

El objetivo de la segunda entrega no es competir con una implementacion GPU de estado del arte, sino construir una version paralela defendible y medir sus costos. Por eso se conserva MPI como solucion principal y se agregan variantes OpenMP para comparar memoria distribuida contra memoria compartida.

2. Problema y algoritmo base

Sea m el numero de triangulos, r la resolucion por eje, $V = r^3$ el numero de voxels y $w = \lceil V/64 \rceil$ el numero de palabras de 64 bits usadas para guardar la grilla bit-packed. Para cada triangulo i , el algoritmo calcula su AABB discreto y visita b_i voxels candidatos. Definimos:

$$B = \sum_{i=1}^m b_i.$$

El trabajo secuencial de la region de voxelizacion es:

$$T_1 = \Theta(m + B).$$

La salida se almacena como una grilla de bits. Marcar un voxel equivale a hacer OR sobre el bit correspondiente.

3. Implementaciones paralelas

3.1. MPI: memoria distribuida

La version MPI reparte triangulos entre procesos con `MPI_Scatterv`. Cada proceso construye una grilla local privada y al final todas las grillas se combinan con `MPI_Reduce` usando `MPI_BOR`.

Esta eleccion evita carreras de datos durante la voxelizacion, porque ningun proceso escribe en la memoria de otro.

Un modelo aproximado del tiempo es:

$$T_p^{MPI} \approx \frac{B}{p} + T_{scatter}(m, p) + T_{reduce}(w, p).$$

La ventaja principal es que el modelo escala a memoria distribuida y hace explicito el costo de comunicacion. La desventaja es que, para problemas pequenos, el overhead de lanzar procesos, distribuir datos y reducir la grilla puede dominar al computo.

3.2. OpenMP con grilla privada

La variante `omp_private` usa un solo proceso y varios threads. Cada thread escribe su propia grilla y al final se hace una reduccion OR en memoria:

$$T_p^{OMP-private} \approx \frac{B}{p} + T_{reduce_mem}(w, p).$$

Esta version evita atomics, pero multiplica la memoria de la grilla por el numero de threads. Es una comparacion natural contra MPI porque conserva la idea de computar resultados privados y combinarlos al final.

3.3. OpenMP con grilla compartida atomica

La variante `omp_atomic` usa una sola grilla compartida. Cada actualizacion al bit-grid se protege con atomic OR sobre la palabra de 64 bits:

$$T_p^{OMP-atomic} \approx \frac{B}{p} + AC_{atomic} + C_{contention}.$$

Aqui A es el numero de actualizaciones y $C_{contention}$ representa el costo extra cuando varios threads intentan modificar la misma palabra del grid. Esta version ahorra memoria, pero puede perder rendimiento si hay mucha contencion.

4. Estado del arte y alcance

Schwarz y Seidel presentan voxelizacion de superficies y solidos en GPU, incluyendo metodos conservadores y estrategias mas avanzadas para cargas grandes. Akenine-Moller propone un test rapido triangulo-caja que puede reducir falsos positivos frente a marcar directamente todo el AABB.

Estas ideas son relevantes, pero cambiarlas por completo excede el alcance de esta entrega. Una version tile-based podria asignar regiones disjuntas de la grilla a cada trabajador y eliminar la reduccion final, pero sin una estructura espacial cada tile tendria que revisar demasiados triangulos. En forma ingenua, el costo se acerca a:

$$T_{tile-naive} = \Theta(r^3 m/p).$$

Para que tile-based sea competitivo se necesita filtrado espacial, por ejemplo binning, BVH, octrees o grillas sparse. CUDA tambien es atractivo para voxelizacion, pero requiere kernels, transferencias host-device, manejo de atomics o reducciones por bloque y validacion contra la version CPU. Por tiempo y defendibilidad, esta entrega prioriza MPI y OpenMP.

5. Metodologia experimental

Se mide la region `voxel_seconds`. Para cada modo, el baseline T_1 es el tiempo del mismo modo con $p = 1$. Las metricas usadas son:

$$S(p) = \frac{T_1}{T_p}, \quad E(p) = \frac{S(p)}{p}, \quad O(p) = pT_p - T_1, \quad R_O(p) = \frac{O(p)}{T_1}.$$

Los experimentos locales usaron la malla `models/blocky_creeper_like.obj`, resoluciones 32, 64 y 128, $p \in \{1, 2, 4\}$ y tres repeticiones por configuracion. Se reporta la mediana para reducir ruido.

6. Resultados locales

Cuadro 1: Resultados locales con resolucion 128 y mediana de tres corridas.

Modo	p	T_p (s)	$S(p)$	$E(p)$	O/T_1
MPI	1	0.002703	1.000	1.000	0.000
MPI	2	0.001755	1.540	0.770	0.299
MPI	4	0.002288	1.181	0.295	2.386
OpenMP atomic	1	0.003745	1.000	1.000	0.000
OpenMP atomic	2	0.002837	1.320	0.660	0.515
OpenMP atomic	4	0.002061	1.817	0.454	1.201
OpenMP private	1	0.003016	1.000	1.000	0.000
OpenMP private	2	0.002748	1.098	0.549	0.822
OpenMP private	4	0.002948	1.023	0.256	2.910

Para resoluciones 32 y 64, los tiempos son muy pequenos y el overhead domina. En resolucion 128 aparece speedup en algunas configuraciones, pero no de forma monotona. MPI obtiene su mejor punto local en $p = 2$, mientras que OpenMP atomic mejora hasta $p = 4$ en esta malla pequena. OpenMP private evita atomics, pero paga el costo de combinar grillas privadas y no supera claramente a las otras variantes en este caso.

7. Validacion en Khipu

Se valido que el proyecto compila y ejecuta en Khipu con Slurm, OpenMPI y CMake, sin depender de Assimp gracias al lector OBJ de respaldo. La corrida conservadora en la particion de debug uso hasta cuatro CPUs y resoluciones pequenas. Esta prueba se trata como validacion de portabilidad y del flujo de ejecucion, no como benchmark definitivo, porque los tiempos son del orden de microsegundos o milisegundos y por tanto muy sensibles al ruido del entorno.

Durante la validacion se observo que el entorno OpenMPI usado no funcionaba correctamente con `srun` directo para lanzar los ranks. La forma estable fue reservar recursos con Slurm y ejecutar el programa MPI con `mpirun` dentro de la asignacion.

8. Discusion

Los resultados muestran que aumentar p no siempre reduce el tiempo. En escalabilidad fuerte el problema se mantiene fijo, por lo que el computo local baja aproximadamente como B/p , pero los costos de reduccion, sincronizacion y lanzamiento no desaparecen. Por eso existe un p optimo: despues de cierto punto, agregar mas procesos o threads aumenta el overhead mas de lo que reduce el computo.

MPI es defendible para este proyecto porque separa claramente memoria distribuida, computo local y reduccion global. Tambien evita carreras de datos sin usar locks ni atomics durante la voxelizacion. OpenMP es mas simple y puede tener menor overhead en una sola maquina, pero no escala a multiples nodos y cada variante tiene un tradeoff: `omp_private` usa mas memoria y `omp_atomic` puede sufrir contencion.

La principal debilidad del particionamiento por triangulos es el balance de carga. Si una malla tiene pocos triangulos gigantes, algunos trabajadores pueden recibir AABBs mucho mas caros que otros. En ese escenario, un enfoque por tiles o una estructura espacial podria balancear mejor, pero esa mejora cambia el nucleo del proyecto y requiere mas implementacion.

9. Conclusiones

La implementacion MPI se mantiene como solucion principal porque representa memoria distribuida de forma directa, evita data races con grillas locales y usa una reduccion OR natural para combinar resultados. Las variantes OpenMP permiten responder por que no basta con decir que memoria compartida es siempre mejor: una version compartida necesita atomics y una version privada paga memoria y reduccion.

Experimentalmente, para la malla pequena usada, el mejor p no es el maximo probado. Esto confirma que el overhead es una parte central del analisis. Para problemas mas grandes, se espera que el computo amortice mejor los costos paralelos, pero para resoluciones pequenas la version secuencial o con pocos workers puede ser suficiente.

Como trabajo futuro quedan tres mejoras principales: reemplazar el marcado AABB por un test triangulo-caja mas preciso, implementar particionamiento tile-based con filtrado espacial, y evaluar una version CUDA cuando se pueda medir en GPU de forma controlada.

10. Referencias

Las referencias completas y las notas de uso de fuentes externas estan en `report/REFERENCES.md`. Las fuentes principales son el trabajo de Schwarz y Seidel sobre voxelizacion en GPU, el test triangulo-caja de Akenine-Moller y la documentacion oficial de MPI/OpenMP/CMake/Assimp.